

AI GPU 方向组会进展

从 GPU Trace Compression 收敛到 AI Decode 下的 Warp Scheduling 分析

dyf

2026-04-05

背景与问题重定义

当前的研究问题已经收敛，不再是“单纯做 trace 压缩”。

新的主线：

difftest 风格的压缩与验证 → 保留 warp-level 行为语义的 GPU trace 表示 → AI workload 中 w

为什么要这样收敛：

- 只讲压缩率，体系结构贡献不够硬
- 只做工具链，很容易停留在工程优化
- 如果 trace 表示能保留控制结构、重复结构和跨层差分，就有机会支撑后续调度分析与优化

核心问题：

- 这种分层表示是否保留了足够的 warp-level 语义
- 是否能帮助定位 AI decode 中 attention / KV-cache 引发的调度问题

研究主线与当前定位

目前判断：warp scheduling 比单纯做 memory trace 更适合作为体系结构切入点。

原因：

- dense matmul / GEMM 通常更规则，调度效应容易被算力吞吐掩盖
- LLM inference, 尤其是 decode 阶段的 attention + KV-cache, 更容易暴露：
 - warp 间推进不均衡
 - memory stall 长尾
 - ready warp 与高收益 warp 不一致
- 当前压缩结构中已有的信息天然接近调度语义：
 - shared PC sequence
 - warp diff
 - address delta / override
 - run-length / sequence folding

策略：

- 先做解释型：定位现有 scheduler 的失效模式
- 再看是否自然长出一个小而硬的调度机制

当前采用的压缩手段

当前不是单一压缩算法，而是一套分层编码策略。

类别	当前采用的手段
Delta encoding	PC delta; threadblock 间 base + offset + override; 跨对象只存变化部分
Default elision	active mask 为全活跃时省略; predicate 与 active 相等时省略
Run-length / sequence compression	连续重复指令段折叠为 instruction run; sequence + run 结构展开恢复
Cross-entity deduplication	跨 warp 共享 PC skeleton; 跨 threadblock 共享 base, 只保留局部 override

如何理解这套压缩：

- 它不是单纯减少字节数
- 更准确地说，是把原始 NVTrace 中显式时间展开的执行语义，提升为以控制结构、重复结构和跨层差分为核心的层次化表示

当前已完成工作与测试结果

1. 压缩与恢复链路

- 已实现 $v4 \leftrightarrow v5$ / $v6$ / $v7$ / $v8$ 的核心编码与解码
- `test_roundtrip` 当前结果: **9 passed, 0 failed**

2. 当前已能明确给出的压缩比

- $v4 \rightarrow v5$ 合成 threadblock 单元测试:
- 原始大小: 258 B
- 压缩后大小: 132 B
- 压缩后为原始大小的 **51.16%**
- 等价压缩倍数约 **1.95x**

3. 模拟器接入

- `parser` 主链路已开始接入 $v5$ / $v6$
- `trace_driven` 中已加入版本分发逻辑
- 还缺: $v7$ / $v8$ `parser` 集成与真实 `trace` 等价性验证

4. 需要说明的限制

AI Trace 方向的最小实验闭环

当前已经决定的 AI workload 路线：

- 不先做 training，先做 inference
- 不先做 full forward，先做 decode
- 不先做大模型，先用 GPT-2 small 打通链路
- 主研究对象先放在 attention + KV-cache
- matmul / GEMM 保留为后续对照组

已经新增的实验脚手架：

- `experiments/gpt2_decode/run_decode.py`
- `experiments/gpt2_decode/run_trace.sh`
- `experiments/gpt2_decode/summarize_runs.py`

第一轮实验设置：

- 模型：GPT-2 small
- 模式：decode only
- `batch = 1`
- `gen_tokens = 1`

为什么 Squash / Batch 适合和 AI 读 Trace 结合

目标：让 AI 能从 trace 中定位问题，而不是只做离线存储压缩。

两者在后续 AI 定位问题中的分工：

- Squash：
 - 把重复执行段、等价局部行为压成“逻辑片段”
 - 相当于把 raw trace 切成 AI 可读的 semantic chunks
- Batch：
 - 把细粒度事件提升到 warp / threadblock / kernel 粒度组织
 - 相当于给 AI 提供可逐层下钻的 hierarchical context

可以形成的定位链：

Kernel summary → Threadblock summary → Warp episode → 可疑长尾 / stall / divergence 片段

预期功能：

- AI 先定位最可疑 kernel
- 再下钻到具体 TB / warp

4 个月投稿窗口下的阶段计划

阶段 1：打通链路并验证可行性

- 完成 GPT-2 decode trace 生成
- 跑通汇总分析
- 确认 trace 中存在可稳定观测的 warp-level 结构变化

阶段 2：形成解释型结果

- 从 attention / KV-cache trace 中提取 warp scheduling 相关特征
- 与 matmul 做对照
- 找出一个稳定、可解释的 scheduler 失效模式

阶段 3：寻找设计机会

- 不强行做大机制
- 只针对一个核心问题，探索小而硬的 scheduler 改进

阶段 4：论文化收敛

- 如果设计机会成立：解释型 + 设计型组合贡献
- 如果设计机会不足：至少形成一篇以体系结构 insight 为核心的解释型论文

当前风险与总结

当前风险：

- decode ROI 中 attention kernel 可能还不够干净
- GPT-2 small 可能过于简单，不足以暴露真实 scheduler 失效模式
- 真实 workload 上 v5-v8 的压缩比与等价性验证还未补齐
- 如果只有解释型结果，没有设计点，体系结构投稿力度可能不足

当前一句话总结：

我们正在把“GPU trace 压缩”收敛为一套借鉴 difftest 思想、可保留 warp-level 行为语义的层次化 trace 表示，并尝试用它分析 LLM decode 中 attention / KV-cache 引发的 warp scheduling 失效模式，进一步寻找体系结构上的调度优化机会。